



第3课-Triton基础-动手写Matmul算子

配套课程

[第3课-Triton基础-动手写Matmul算子.mkv](#)

前言

我们知道，矩阵乘法是线性代数中的基本操作之一，它的定义如下：**当给定两个矩阵 A 和 B**，其中 A 的维度为 $(M \times K)$ ，B 的维度为 $(K \times N)$ ，其中K被称为一个规约轴，则它们的乘积 $C = A \times B$ 的维度为 $(M \times N)$ ，且 C 中的元素 $C[i][j]$ 由以下公式计算：

$$C[i][j] = \sum_{k=1}^K A[i][k] \times B[k][j]$$

矩阵乘法初步

以下是一个简单的matmul实现，用三重循环实现。

代码块

```
1  def matrix_multiply(A, B):
2      # A B 都是二维列表
3      A_shape = A.shape
4      B_shape = B.shape
5      rows_A = A_shape[0]
6      cols_A = A_shape[1]
7      rows_B = B_shape[0]
8      cols_B = B_shape[1]
9      assert cols_A == rows_B
10     C = np.zeros((rows_A, cols_B))
11     for i in range(rows_A):
12         for j in range(cols_B):
13             for k in range(rows_B):# 作为一个规约轴
14                 C[i][j] += A[i][k] * B[k][j]
15
16     return C
```

在进行矩阵乘法时，通常需要使用三重嵌套循环来正确组合矩阵元素。实现通用矩阵乘法（GEMM）时，我们需要在输出矩阵的M轴（行）和N轴（列）上迭代，同时在规约轴K上共同迭代。在矩阵乘法中，共同迭代指的是对规约轴（K轴）的累加操作。在上方的矩阵乘法代码中，M轴等于rows_A，N轴等于cols_B，规约轴K轴等于rows_B。

总的来说，如果将K轴放在最内层循环，则每次计算输出矩阵的一个元素时，会遍历K轴并进行累加。这种方式的局部性较好，适合缓存优化。如果将M轴或N轴放在最内层循环，则可能会导致频繁的内存访问和缓存miss，如此一来就会降低计算效率。另外，改变M和N的循环顺序也可以产生两种不同的计算模式。

分块矩阵乘法

虽然经验证，上述矩阵乘法函数结果正确，但是很明显这样的实现性能是不够的。可以考虑**分块矩阵乘法**（Tiled Matrix Multiplication），即将两个输入的矩阵按照一定的大小进行分块，每次计算一部分内容。分块的目的就是优化访存，通过分块之后让访存尽可能地集中在一定区域内，从而能够提高数据局部性，从而提高 Cache 利用率，性能就会更好。

假设现在进行矩阵相乘的两个矩阵A和B，维度依次为(32, 64)和(64, 128)，我们选的分块大小为8，也就是说A矩阵会被分为一个2×4个小块（也就是行方向上分为2块，列方向上分为4块），每个小块的维度等于16×16，B矩阵会被分为4×8个小块。分块矩阵乘法的代码和示意图如下所示：

代码块

```
1 def matrix_multiply_blocked(A, B, BLOCK_SIZE):
2     M, K = A.shape
3     K, N = B.shape
4
5     C = np.zeros((M, N), dtype=np.float32)
6     # 在m轴上遍历块
7     for m in range(0, M, BLOCK_SIZE):
8         # 在n轴上遍历块
9         for n in range(0, N, BLOCK_SIZE):
10            acc = np.zeros((BLOCK_SIZE, BLOCK_SIZE),
11                           dtype=np.float32)
12            # 在k轴上遍历块
13            for k in range(0, K, BLOCK_SIZE):
14                a = A[m: m + BLOCK_SIZE, k: k + BLOCK_SIZE]
15                b = B[k: k + BLOCK_SIZE, n: n + BLOCK_SIZE]
16
17                acc += np.dot(a, b)
18
19            C[m: m + BLOCK_SIZE, n: n + BLOCK_SIZE] = acc
20
21     return C
```

在K轴规约过程中，矩阵乘法针对的小块a和b，它们的尺寸均为[BLOCK_SIZE, BLOCK_SIZE]，并通过np.dot函数计算这两个小块的乘积。每次完成K轴的规约后，我们将多个小块的乘积结果累加到累加器acc中，该累加器的值等于矩阵A的一行与矩阵B的一列的乘积结果。

代码块

```
1 Matrix A (32x64), block size 16:
2
```

| acc += np.dot(a, b)



张辉 2025年10月9日

用 np.dot，比较出的时间不就没意义了么，快是快在 numpy 上，这里公平比话按理说也得用 matrix_multiply

3	+-----+-----+-----+-----			
	-+			
4				
5		16x16		16x16
				16x16
6				
7	+-----+-----+-----+-----			
	-+			
8				
9		16x16		16x16
				16x16
10				
11	+-----+-----+-----+-----			
	-+			
12				
13	Matrix B (64x128), block size 16:			
14				
15	+-----+-----+-----+-----			
	-+			
	--+			
16				
17		16x16		16x16
		16x16		16x16
18				
19	+-----+-----+-----+-----			
	-+			
	--+			
20				
21		16x16		16x16
		16x16		16x16
22				
23	+-----+-----+-----+-----			
	-+			
	--+			
24				
25		16x16		16x16
		16x16		16x16
26				
27	+-----+-----+-----+-----			
	-+			
	--+			
28				

另外，分块矩阵乘法利用了数据的空间局部性，即访问的数据附近的数据也有较高的访问概率。通过分块，可以确保每次计算所需的数据在缓存中连续存储，从而减少了缓存不命中的次数

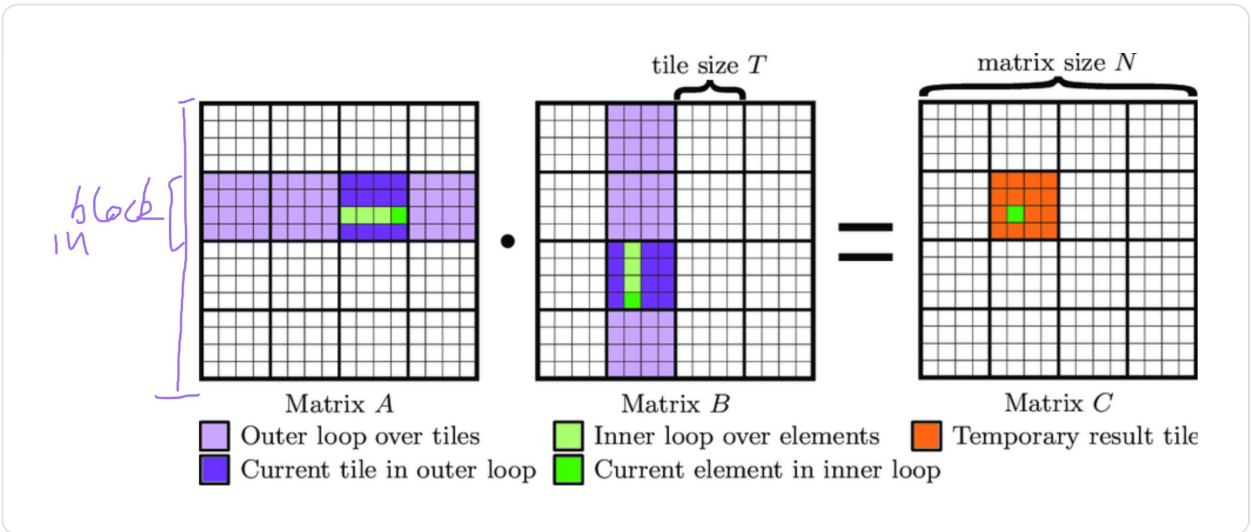
用Triton实现分块矩阵乘法

在前文中我们已经了解到，Triton 以 **Block** 为基本计算单位进行并行处理。为了高效实现分块矩阵乘法（Tiled Matrix Multiplication），我们需要合理地配置 Kernel 的执行网格（grid）结构。

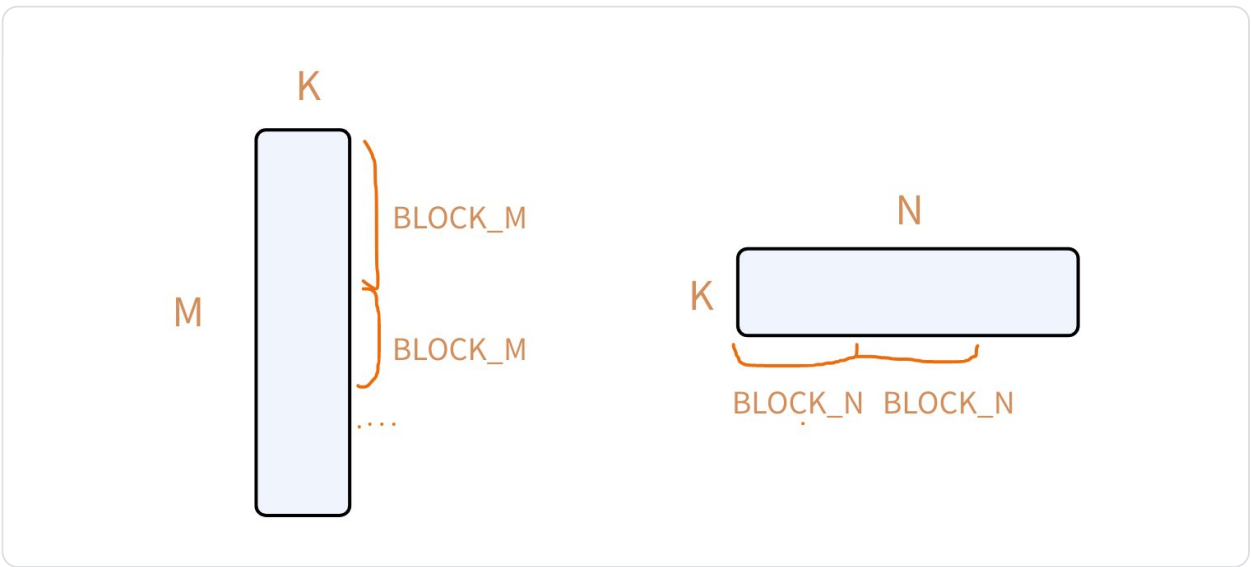
具体来说，我们将构建一个**二维网格**：

- 网格的第一个维度（即 X 轴）对应于输出矩阵在 **M 轴** 上的分块数量；
- 第二个维度（即 Y 轴）对应于输出矩阵在 **N 轴** 上的分块数量。

看下面我的草图：假设我们为M轴的遍历准备了每次的大小是BLOCK_M，总共需要在网
格当中配置M/BLOCK_M个BLOCK。



M和N轴上的分块



1. 矩阵乘法的基本设定

我们要计算的是两个矩阵相乘：

- A：形状为 (M, K)
- B：形状为 (K, N)
- 结果 C：形状为 (M, N)

2. 分块策略概述

为了提升性能，我们将整个矩阵划分成多个小块（tiles），每个 Triton block 处理一对输入块的乘积，并累加到输出矩阵的相应位置。

- BLOCK_M：每个 Triton block 负责的 M 维度上的大小。
- BLOCK_N：每个 Triton block 负责的 N 维度上的大小。

- BLOCK_K: 通常固定为 32 或 64, 用于对 K 维度进行分段处理 (不是本段重点)。

3. Triton Block 的编号与负责区域

GPU 上会启动多个 Triton block 并行执行, 每个 block 在二维网格中有自己的编号:

- `pid_m`: 表示该 block 是第几个处理 M 维度的块。
- `pid_n`: 表示该 block 是第几个处理 N 维度的块。

这两个编号共同决定了当前 block 负责的输出矩阵部分。

每个 block 负责的范围:

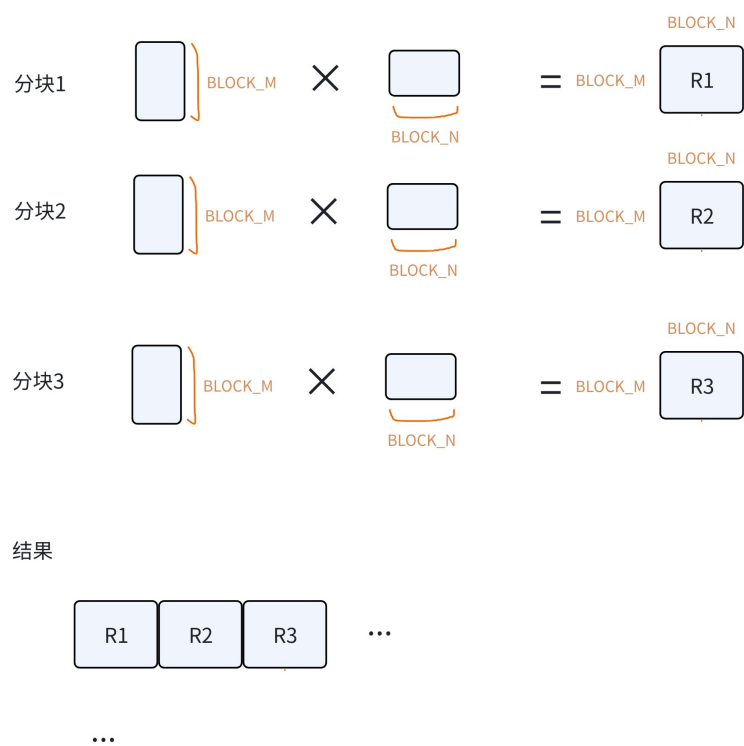
- M 方向: 起始 = `pid_m * BLOCK_M`, 结束 = `pid_m * BLOCK_M + BLOCK_M`
 - 即 `offs_m = [pid_m * BLOCK_M : pid_m * BLOCK_M + BLOCK_M]`
- N 方向: 起始 = `pid_n * BLOCK_N`, 结束 = `pid_n * BLOCK_N + BLOCK_N`
 - 即 `offs_n = [pid_n * BLOCK_N : pid_n * BLOCK_N + BLOCK_N]`

因此, 该 block 最终负责计算输出矩阵 C 中大小为 `BLOCK_M * BLOCK_N` 的一个子块, 它的坐标记作 `C[offs_m, offs_n]`。

如下图所示的这种分块方式, 使得每个块可以独立地处理其负责的矩阵分块, 最后的结果就是这些矩阵分块的拼接, 从而提高了计算的并行性和缓存利用率。按照上面的叙述, 我们写出部分的Triton代码:

代码块

```
1  @triton.jit
2  def _fused_linear_kernel_fwd(
3      x_ptr,
4      r_ptr=None,
5      apply_gelu=False,
6      seed=1337,
7      BLOCK_SIZE_M: tl.constexpr = 128,
8      BLOCK_SIZE_K: tl.constexpr = 64,
9  ):
10
11      offs_m = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)[:],
12      None]
13      offs_n = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
14      [None, :]
15      z = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
```



K轴上的规约

在确定了当前Block负责的M轴和N轴范围后，我们需要在K轴上进行滑动和规约操作。具体步骤如下：

- M轴范围：当前Block负责的M轴范围为 $[pid_m \times BLOCK_M, pid_m \times BLOCK_M + BLOCK_M)$ ，记作 **offs_m**。
- N轴范围：当前Block负责的N轴范围为 $[pid_n \times BLOCK_N, pid_n \times BLOCK_N + BLOCK_N)$ ，记作 **offs_n**。
- K轴范围：K轴的范围是从 0 到 K，每个分块的大小为 $BLOCK_K$ 。我们需要在K轴上滑动，每次处理一个大小为 $BLOCK_K$ 的分块。

综上，每个分块的计算结果是一个大小为 $BLOCK_M \times BLOCK_N$ 的矩阵，记作 z 。初始时， z 被初始化为零矩阵。在K轴上，我们通过滑动分块的方式逐步计算矩阵乘法。具体来说：从K轴的起点开始，每次取出一个大小为 $BLOCK_M \times BLOCK_K$ 的 a 块和一个大小为 $BLOCK_K \times BLOCK_N$ 的 b 块。

将 a 块和 b 块进行矩阵乘法，并将结果累加到 z 中。当K轴上的所有分块都被处理完成后， z 中存储的就是当前Block负责的 $M \times N$ 范围内的最终计算结果。因此我们有下面的K轴规约代码：

代码块

```
1  z = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
2  for k in range(0, K, BLOCK_SIZE_K):
3      x_k = tl.arange(0, BLOCK_SIZE_K)[None, :] + k
4      x = tl.load(x_ptr + offs_m * K + x_k, mask=(offs_m < M) & (x_k
5      < K), other=0.0)
6      x = x.to(tl.float16)
7      w_k = tl.arange(0, BLOCK_SIZE_K)[: , None] + k
8      w = tl.load(w_ptr + w_k * N + offs_n, mask=(w_k < K) & (offs_n
9      < N), other=0.0)
10     w = w.to(tl.float16)
11     z = tl.dot(x, w, acc=z)
```

其中 x , w 分别表示我们上文中的 a , b 小块，它们在K轴上的所有分块依次进行矩阵相乘($tl.dot$)并将结果累加到 z 上。

结果的写回

我们需要将数据写回的位置为(`offs_m`, `offs_n`)，其中`offs_m`表示的范围是 $[\text{pid_m} \times \text{BLOCK_M}, \text{pid_m} \times \text{BLOCK_M} + \text{BLOCK_M})$ ，而`offs_n`表示的范围是 $[\text{pid_n} \times \text{BLOCK_N}, \text{pid_n} \times \text{BLOCK_N} + \text{BLOCK_N})$ ，它们共同表示结果子矩阵写回的地址范围。同时，`mask`用于限制我们写回的范围不超过`M`和`N`。

代码块

```
1  z_offset = offs_m * N + offs_n
2  z_mask = (offs_m < M) & (offs_n < N)
3
4  tl.store(z_ptr + z_offset, z, mask=z_mask)
```

kernel启动的配置

代码块

```
1  z = torch.empty((M, N), device=x.device, dtype=x.dtype)
2
3  BLOCK_SIZE_M = 64
4  BLOCK_SIZE_N = 64
5  BLOCK_SIZE_K = 32
6
7  # 2D launch kernel where each block gets its own program.
8  grid = (triton.cdiv(M, BLOCK_SIZE_M), triton.cdiv(N,
    BLOCK_SIZE_N), 1)
9  _fused_linear_kernel_fwd[grid](
10     x,
11     weight,
12     z,
13     M, N, K,
14     BLOCK_SIZE_M=BLOCK_SIZE_M,
15     BLOCK_SIZE_N=BLOCK_SIZE_N,
16     BLOCK_SIZE_K=BLOCK_SIZE_K,
17 )
```

我们为`M`轴、`N`轴和`K`轴分别设定了不同的分块大小，并配置了一个二维网格`grid`。其中，第0维和第1维的`BLOCK`数量分别为 `(M/ BLOCK_SIZE_M)` 和 `(N/ BLOCK_SIZE_N)`，这与上文的分析完全一致。同时，我们会额外为矩阵乘准备了一个张量`z`，用于保存矩阵相乘的输出。

Triton矩阵乘法结果验证

代码块

```
1  ...
2
3  if __name__ == '__main__':
4      batch_size = 4
5      sequence_length = 10
6      hidden_dim = 128
7
8      # 假设权重矩阵 weight 的形状为 [hidden_dim, output_dim]
9      output_dim = 256
10
11     x = torch.randn((batch_size, sequence_length, hidden_dim),
        device='cuda', dtype=torch.float16)
```



```
12     weight = torch.randn((hidden_dim, output_dim), device='cuda',
13                             dtype=torch.float16)
14     output = fused_ffn(x, weight)
15     golden = x@weight
16     print(output.shape)
17     print(golden.shape)
```

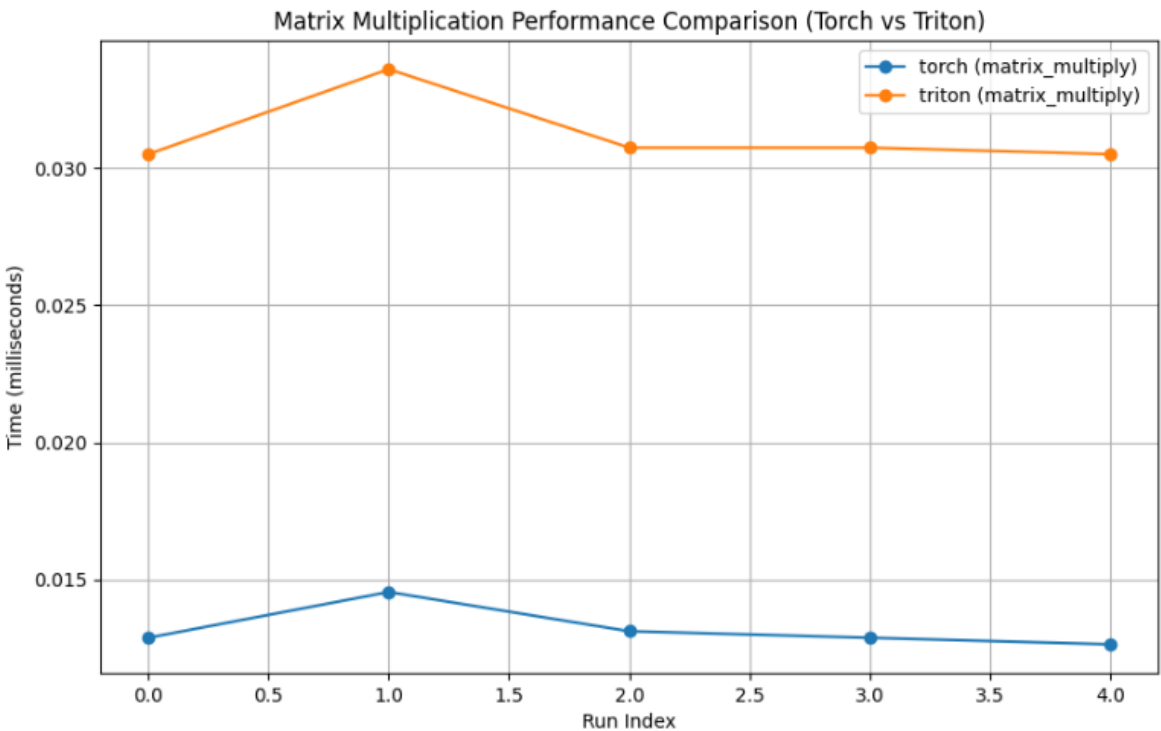
我们准备的两个输入x和w的维度分别为 (batch_size, sequence_length, hidden_dim) 和 (hidden_dim, out_dim)，其中 hidden_dim 所在的轴是它们的规约轴，也就是说矩阵A的维度等于 (batch_size × seq_length, hidden_dim)，矩阵B的维度等于 (hidden_dim, out_dim)。

通过比较Triton矩阵乘的输出结果output与Pytorch矩阵乘的结果，我们发现两者完全一致（差异极小）。另外，我们也将测试Triton Kernel和Pytorch矩阵乘的速度差异，可以看到我们实现的算子还是和Pytorch有一定的速度差异，但是已经在同一个数量级上。

代码块

```
1  for i in range(repeat_time):
2      # 重新生成输入
3      x = torch.randn((batch_size, sequence_length, hidden_dim),
4                      device='cuda', dtype=torch.float16)
5      weight = torch.randn((hidden_dim, output_dim), device='cuda',
6                          dtype=torch.float16)
7
8      t1 = time.time()
9      output = fused_ffn(x, weight)
10     t2 = time.time()
11     # print('triton time:{}'.format(t2 - t1))
12     times_triton.append(t2 - t1)
13
14     t1 = time.time()
15     golden = x@weight
16     t2 = time.time()
17     times_torch.append(t2-t1)
18     print('pytorch time:{}'.format(t2 - t1))
19     print(output.shape)
20     print(golden.shape)
```

我们在验证时，矩阵乘法使用的两个输入维度分别为8192×1280和1280×2560。



t1 = time.time() output = fused_ffn(x, w...



王飞扬 4月14日 10:53

需要加上
torch.cuda.synchronize()

我们在验证时，矩阵乘法使用的两个输入...



刘涵祚 2025年9月1日

这里我验证的 triton 更快啊, 时间大概是 0.75-0.85 倍的 torch 左右



ArdenX 3月6日 15:45

不同硬件对于设置的 block 反映不同，可能这设置刚好对你的硬件比较好

后期通过参数的autotune(自动调优)，针对BLOCK_M,BLOCK_N,BLOCK_K，差距会更近。